Linked list — everything written down

Labs, tutorials, the Linked Lis practice files, and the patterns that exist only in linked_list_notes.md. Updated 23 Sep 2026.

The split complete-solution file is the same index split. Doubly insert is written twice. Same links. The dictionary cycle check is the other way to measure a cycle. Same job as Floyd.

Singly — data and next

Find node	walk index steps from head.
Insert at index	index 0: new node points at the old head. Later: find the node before, link around the new node.
Insert at back	not a new pattern. Walk until cur.next is None, then cur.next = new. Circular: stop when cur.next is head, then new.next = head and last.next = new. Head stays.
Remove at index	need the node before the one you drop. prev.next = cur.next. Index 0: head = head.next.
Remove duplicates, sorted	if this equals next, skip next. Else move on.
Split by index	counter. Index 0, 2, 4 in one list. 1, 3, 5 in the other. This is the position, not the value.

find node

```
cur = self.head
while i > 0:
    cur = cur.next
    i -= 1
return cur
```

Doubly — same jobs, and prev

Insert at index	index 0: new.next = head, and head.prev = new. Later: new sits between current and current.next. Fix both sides.
Delete by index	index 0: head = head.next, and the new head's prev is None. Later: current.prev.next = current.next, and the other way too.
Delete by value	walk until the value. Same unlink. If it is the head, move head.

insert and find check if i < 0 or i > ll.size

insert

1. if i == 0:

```
new_node.next = head
if head:                    # only for doubly, check if head even existed
    head.prev = new_node
head = new_node
```

2. else (middle of end)

```
cur = self.head
while i-1 > 0:
    cur = cur.next
    i -= 1
# this stops 1 before the insert position
new_node.next = cur.next
new_node.prev = cur         # only for doubly
if cur.next:                # only for doubly, check if its the end
    cur.next.prev = new_node
cur.next = new_node
```

remember update size

deletion

1. by index to the same as find node

```
if self.head is None:
    return None
if i < 0 or i >= self.size:
    reutrn None
if i == 0:
    self.head = self.head.next
    if self.head:  # If the list is not empty after removal
        self.head.prev = None
else:
    while i > 0:
        cur = cur.next
        i -= 1
    cur.prev.next = cur.next
    if cur.next:  # If it's not the last node
        cur.next.prev = cur.prev
```

remember update size

2. by number just traverse (while current) and see if data ==

```
while current:
    if current.data == data:
        if current.prev:   # checking if it is head
            current.prev.next = current.next
        else:              # deleting head
            self.head = current.next
        if current.next:   # chekcing if not the last
            current.next.prev = current.prev
        self.size -= 1
        return True

    # traverse the whole thing
    current = current.next
```

Circular — last node points back at the first. No None.

Singly	last.next = head. Keep walking forward and you come back.
Doubly	last.next = head, AND head.prev = last. No None either way.

Display / search / size: stop when you are back at head, not at None.

```
cur = head
while cur.next is not head:
    cur = cur.next
# cur is the last node. cur.next is head.
# insert at back uses this same stop, then new.next = head and last.next = new
```

Insert / delete: if the first or last node changes, update head AND the link between first and last. Doubly: fix both directions (neighbours, the circular links, and head).

Singly. last is not stored. Find it: walk until cur.next is head. Middle does not touch head. Same splice as a normal list.

insert at head:

```
if head is None:          # empty
    new.next = new
    head = new
else:
    last = head
    while last.next is not head:
        last = last.next
    new.next = head
    last.next = new
    head = new
```

insert middle (index i > 0):

```
prev = head
for _ in range(i - 1):
    prev = prev.next
new.next = prev.next
prev.next = new
```

delete head:

```
if head.next is head:     # only node (becasue cirualr means point back to itself)
    head = None
else:
    last = head
    while last.next is not head:
        last = last.next
    last.next = head.next
    head = head.next
```

delete middle (index i > 0):

```
prev = head
for _ in range(i - 1):
    prev = prev.next
prev.next = prev.next.next
```

Doubly. head.prev IS the last node, so no walk.

insert at head:

```
last = head.prev
new.next = head
new.prev = last
last.next = new
head.prev = new
head = new
```

delete head:

```
last = head.prev
head = head.next
last.next = head
head.prev = last
```

Advantage: keep cycling. Round-robin: each one gets a turn, then back to the first. Disadvantage: wrong stop = infinite loop. Circular buffer is the same idea, but it is not always a linked list.

Reverse — save next, point back, advance

Reverse doubly	same loop, and prev is set to the old next.
List palindrome	this same loop, started from the middle. Finding the middle and the compare are under slow/fast.

use 3 pointers

```
prev = None
cur = head

while cur:
    next_node = cur.next
    cur.prev = next.node   # only for doubly linked
    cur.next = prev
    prev = cur
    cur = next_node

return prev   # prev is the new head
```

memory trick: "save next, flip, advance both"

- save next (so you don't lose the rest of the list)
- point cur.next backward to prev
- shift prev and cur one step forward

Slow / fast — fast moves two steps

Cycle length	when they meet, walk the loop and count. Other way, in the notes and commented in the file: save each node in a dict, return when you see it again.
Delete middle	same walk, and keep prev. Then prev.next = slow.next. One node: head = slow.next. Written in the notes. There is no linked-list file for it. The delete-middle file is a stack.
List palindrome	slow/fast finds the middle. Then reverse the second half. Then compare.

```
prev = None
slow = fast = head
while fast is not None and fast.next is not None:
    prev = slow
```

```
    slow = slow.next
    fast = fast.next.next
```

Cycle detection

move first, then compare. they start on the same node. return 0 only after the loop, if they never meet.

```
slow = fast = head
while fast is not None and fast.next is not None:
    slow = slow.next
    fast = fast.next.next
    if slow is fast:   # is means same object
        return True
return 0               # outside the loop
```

Cycle lenght count

countinue on

```
lenght = 1
cur = slow.next
while cur is not slow:
    cur = cur.next
    lenght += 1
return lenght
```

Palidrome check

1. When loop stops:

- ODD (e.g. 1→2→3→2→1): slow = exact middle (3). prev = node before middle (2).
- EVEN (e.g. 1→2→2→1): slow = 1st node of 2nd half (2nd 2). prev = last node of 1st half (1st 2) ← one BEFORE 2nd half starts.

Example EVEN 1→2→2→1: after loop → prev=1st 2, slow=2nd 2.

Example ODD 1→2→3→2→1: after loop → prev=2, slow=3.

2. Reverse from slow (same reverse algo, prev starts at None, cur starts at slow)

```
prev = None
cur = slow

while cur:
    next_node = cur.next
    cur.next = prev
```

```
    prev = cur
    cur = next_node
```

3. compare front vs prev

```
front = head
while prev is not None:   # check on prev because of the even cases
    if prev.data != front.data:
        return False
    prev = prev.next
    front = front.next
return True
```

Why prev = None when reversing? NOT because we stopped in middle — any reverse starts with prev=None because the first node reversed becomes the tail (must point to None).

Delete middle

when loop stops

```
# delete slow
if prev:
    prev.next = slow.next
else:
    head = slow.next    # list had 1 node only
```

- ODD: slow IS the middle → delete it (use prev to skip over it)
- EVEN: slow = start of 2nd half, prev = last of 1st half. delete slow (right-middle) via prev.next = slow.next. prev is the node one before 2nd half — that's your even anchor.

Find node + unlink/relink (move max OR move min to front)

Same pattern for both — only difference is what you search for.

MOVE MAX/MIN:

```
walk list, track max_node AND pre_max (node before max)
if pre_max is None → max already at front, done
else:
    pre_max.next = max_node.next
    max_node.next = head
    head = max_node
```

Remove duplicates (sorted list only!)

Single pass — duplicates are adjacent when sorted.

```
cur = head
while cur.next:
    if cur.data == cur.next.data:
        cur.next = cur.next.next
    else:
        cur = cur.next
```

if not arranged, open dictionary, save number of times appear, then second apss minus off until each become 1

Partition by value

Two different writes of the even/odd split.

Move odds to the back	two chains. Detach each node. Evens, then attach the odds.
Move evens to the back	walk with an index. Remove the even, insert it at the end.

for this cannot use while current becasue you want to cap the loop in a fixed number of runs else it will never finish becaue it will keep moving things to the back

```
index = 0
cur = head
for _ in range (self.size):
    if cur.data & 2 == 0 or != 0
        even_value = cur.data
        cur = cur.next
        ll.removeNode(index)
        ll.insertNode(even_value, ll.size)
        #index dont need change value
    else:
        index += 1
        cur = cur.next
```

if no size is given or remove or insert node, need to create 2 seperate linked list and then join them

```
even_head = None
even_tail = None
odd_head = None
odd_tail = None
cur = head

while cur is not None:
    next_node = cur.next
    cur.next = None
```

```
        if cur.data % 2 == 0:
            if even_head is None:
                even_head = even_tail = cur
            else:
                even_tail.next = cur
                even_tail = cur
        else:
            if odd_head is None:
                odd_head = odd_tail = cur
            else:
                odd_tail.next = cur
                odd_tail = cur
        cur = next_node

    if even_head is None:
        return odd_head
    else:
        even_tail.next = odd_head
        return even_head
```

Merge two sorted lists

Dummy head, then the smaller of the two fronts, and attach whichever list is left.

```
dummy = Node(0)
tail = dummy

while l1 and l2:
    if l1.data <= l2.data:
        tail.next = l1
        l1 = l1.next
    else:
        tail.next = l2
        l2 = l2.next

    # tail is moved ahead for every single time to go to the end
    tail = tail.next

tail.next = l1 if l1 else l2   # attach leftover
return dummy.next
```

Which technique when?

Palindrome?	slow/fast + reverse half + compare
Delete middle?	slow/fast + prev, delete slow
Cycle / cycle length?	Floyd (slow/fast)
Circular?	last.next = head. Stop when back at head. Doubly: head.prev = last too.

Insert at back?	walk until cur.next is None, then cur.next = new. Circular: stop when cur.next is head.
Partition evens/odds?	two sublists + reconnect
Merge sorted lists?	dummy head + compare + attach leftover
Split by index 0,2,4?	counter i % 2
Remove dupes (sorted)?	single pass skip matching next
Move max/min to front?	find node + prev, unlink, prepend
Reverse whole list?	prev/cur/next loop, return prev
Doubly insert/delete?	update BOTH next and prev pointers